

Exemple, ExempleCollection (v2)

Fabrice.Kordon@lip6.fr



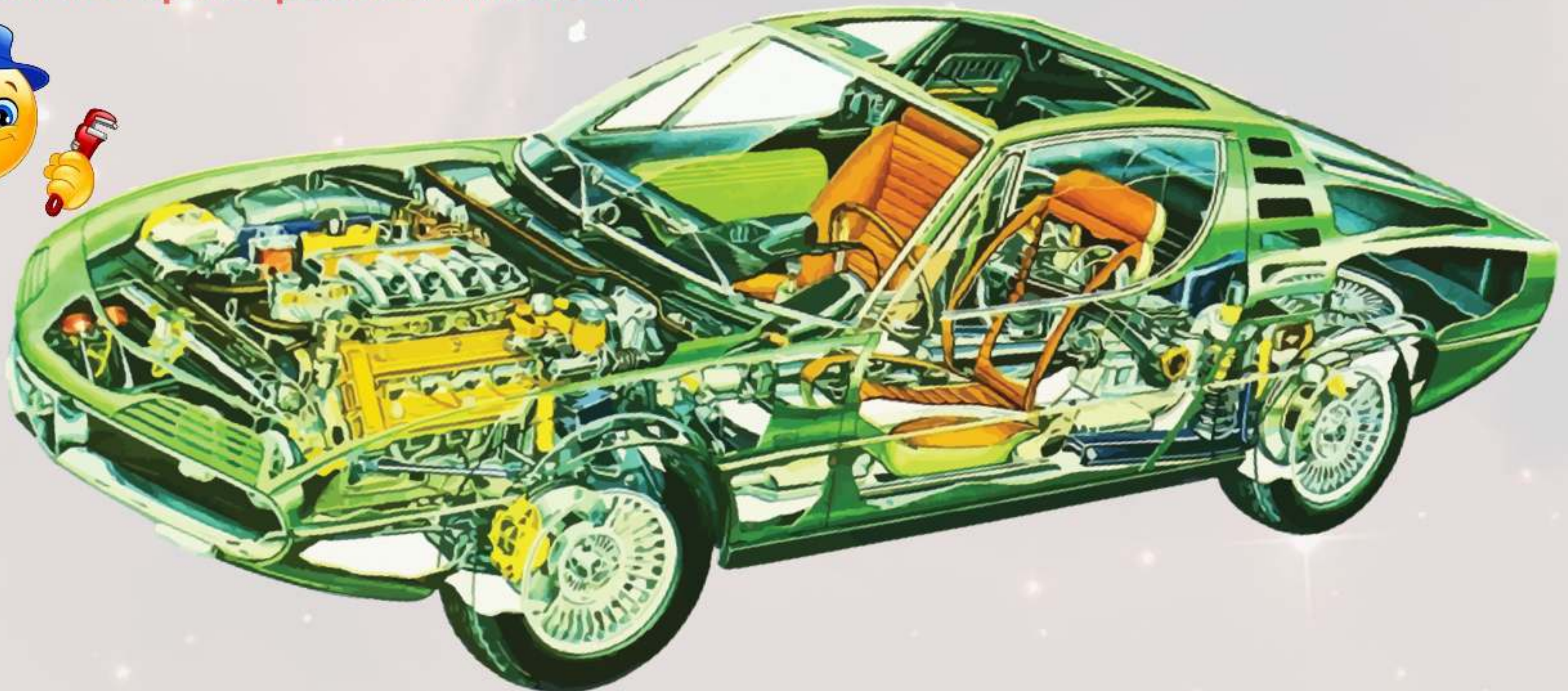
Objectif

2

Montrer la manipulation de collections de threads

- 🔧 Dans le contexte de ReentrantLock...
 - ▶ Afficher les threads bloquées sur une condition

- 🔧 Cela peut-être pratique pour debugger
 - ▶ Voir «ce qui se passe à l'intérieur»



Ce que l'on va programmer

3

Un peu artificiel (pour les besoins de la cause 😄)

- Illustrer l'usage des Génériques
- Manipuler des Threads
- Interroger un ReentrantLock
 - ▶ Observer ceux qui sont bloqués sur une condition

Quelques «utilitaires»

4

```
import java.util.Random;
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.locks.Condition;
import java.util.Collection;
import java.lang.ThreadLocal;
import java.util.ArrayList;

/* Nécessaire car on ne peut utiliser les methodes protegees de
   ReentrantLock que dans une sous-classe de ReentrantLock */
@SuppressWarnings(«serial») // toujours cette histoire de warning
class MyLock extends ReentrantLock {
    public MyLock(boolean t) {
        super(t);
    }

    // redéfinition d'un opérateur générique existant dans ReentrantLock
    Collection<Thread> threadsEnAttente(Condition condition) {
        return super.getWaitingThreads(condition); // l'opérateur existant
    }
}
```


La classe LeBloqueur

5

```
class LeBloqueur {
    private final MyLock l = new MyLock(true); // activer l'équité

    private final Condition c = l.newCondition();

    public void jeMeBloque () {
        l.lock();
        try {
            c.await();
        }
        catch (InterruptedException e) {
            System.out.println ("Ne doit pas arriver (1).");
        }
        finally {
            l.unlock();
        }
    }

    protected void lesBloques() {
        System.out.println ("Voici la liste des bloqués:");
        l.lock();
        // A faire dans un lock/unlock (cf manuel)
        ArrayList<Thread> threadsBloquees = new ArrayList<Thread>(l.threadsEnAttente(c));
        l.unlock();
        for (int i = 0; i < threadsBloquees.size() ; i++) {
            System.out.println ("    " + (i+1) + " -> " + threadsBloquees.get(i));
        }
    }
}
```


La classe UneThread

6

```
class UneThread implements Runnable {
    private int monId;
    private LeBloqueur monBloqueur;

    private static int cpt = 1;
    private static Object mutex = new Object();

    UneThread (LeBloqueur b) {
        monBloqueur = b;
        synchronized(mutex){
            monId = cpt++;
        }
    }

    public int getMonId() {
        return monId;
    }

    private void trace (String m) {
        System.out.println ("E " + monId + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        this.trace("initialisé");
        monBloqueur.jeMeBloque(); //Se bloquer (artificiel)
        this.trace("bye bye");
    }
}
```


La «classe principale»

7

```
public class ExempleCollections2 {
    static int nbBloques;

    public static void main(String []args) {
        try {
            nbBloques = Integer.parseInt(args[0]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, nombre de processus bloqués.");
            return;
        }

        LeBloqueur bloqueur = new LeBloqueur();

        for (int i = 0; i < nbBloques ; i++) {
            UnThread ut = new UnThread(bloqueur);
            new Thread(ut, "UnThread(" + (ut.getMonId()) + ")").start();
        }
        try {
            Thread.sleep(1000);
        }
        catch (InterruptedException e) {
            System.err.println ("Ne doit pas arriver (2).");
        }
        bloqueur.lesBloques(); // interroger le bloqueur
    }
}
```


Quelques exécutions...

8

```
$ java ExempleCollections2 11
```

```
E 1 : initialisé  
E 4 : initialisé  
E 3 : initialisé  
E 2 : initialisé  
E 6 : initialisé  
E 5 : initialisé  
E 7 : initialisé  
E 8 : initialisé  
E 9 : initialisé  
E 10 : initialisé  
E 11 : initialisé
```

Voici la liste des bloqués:

```
1 -> Thread[UneThread(1),5,main]  
2 -> Thread[UneThread(4),5,main]  
3 -> Thread[UneThread(3),5,main]  
4 -> Thread[UneThread(2),5,main]  
5 -> Thread[UneThread(6),5,main]  
6 -> Thread[UneThread(5),5,main]  
7 -> Thread[UneThread(7),5,main]  
8 -> Thread[UneThread(8),5,main]  
9 -> Thread[UneThread(9),5,main]  
10 -> Thread[UneThread(10),5,main]  
11 -> Thread[UneThread(11),5,main]
```

^C

```
$ java ExempleCollections2 5
```

```
E 1 : initialisé  
E 5 : initialisé  
E 4 : initialisé  
E 2 : initialisé  
E 3 : initialisé
```

Voici la liste des bloqués:

```
1 -> Thread[UneThread(1),5,main]  
2 -> Thread[UneThread(4),5,main]  
3 -> Thread[UneThread(5),5,main]  
4 -> Thread[UneThread(2),5,main]  
5 -> Thread[UneThread(3),5,main]
```

^C

En guise de conclusion...

9

Au delà de l'exemple

- Un «truc» utile pour déboguer vos programmes
 - ▶ Inspecter les threads bloquées dans un lock
 - ▶ primitive équivalente pour les conditions...
 - ▶ bref...

